

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5

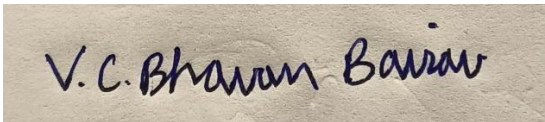
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I V.C.BHAVAN BAIRAV – 192511369 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. COMPLETE QUERY PROCESSING PIPELINE

PROBLEM UNDERSTANDING

The objective is to process an SQL query from input to final output. The pipeline includes parsing, validation, query-tree generation, optimization, execution, and result generation.

ALGORITHM

Algorithm Complete_Query_Processing(Q)

Input: SQL query Q

Output: Result R

1. Receive the SQL query.
2. Perform lexical analysis.
3. Check syntax validity.
4. Validate the query semantically.
5. Generate the query tree.
6. Generate alternative execution plans.
7. Estimate the cost of each plan.
8. Select the plan with minimum cost.
9. Execute the selected plan.
10. Display the result.

PSEUDOCODE

Algorithm Complete_Query_Processing(Q)

Input:

SQL query Q

Output:

Result R

Begin

Receive Q

Tokens $\leftarrow$ Lexical_Analysis(Q)

If Syntax_Is_Invalid(Tokens) Then

 Display "Syntax Error"

 Return

End If

Parsed_Query $\leftarrow$ Parse(Tokens)

If Not Validate_Semantics(Parsed_Query) Then

 Display "Semantic Error"

 Return

End If

Query_Tree $\leftarrow$ Generate_Query_Tree(Parsed_Query)

Plans $\leftarrow$ Generate_Alternative_Plans(Query_Tree)

Best_Plan $\leftarrow$ Null

Min_Cost $\leftarrow$ Infinity

For Each Plan In Plans Do

 Cost $\leftarrow$ Estimate_Cost(Plan)

 If Cost < Min_Cost Then

 Min_Cost $\leftarrow$ Cost

 Best_Plan $\leftarrow$ Plan

 End If

End For

$R \leftarrow \text{Execute}(\text{Best_Plan})$

Display R

End

RESULT

The SQL query is successfully processed through parsing, optimization, and execution to produce the final result.

Q2. COST-BASED QUERY OPTIMIZATION

PROBLEM UNDERSTANDING

Different join orderings can produce the same final result but may have different I/O costs. The objective is to estimate the cost of each possible join ordering and select the plan with the minimum cost.

ALGORITHM

Algorithm Cost_Based_Optimization(Relations)

Input: Relations R1, R2, ..., Rn

Output: Optimal join order

1. Generate possible join orderings.
2. Calculate the estimated I/O cost for each ordering.
3. Estimate intermediate relation sizes.
4. Compare the total costs.
5. Select the ordering with minimum cost.
6. Return the optimal plan.

PSEUDOCODE

Algorithm Cost_Based_Optimization(Relations)

Input:

Relations R1, R2, ..., Rn

Output:

Optimal_Join_Order

Begin

Plans $\leftarrow$ Generate_Join_Orderings(Relations)

Best_Plan $\leftarrow$ Null

Min_Cost $\leftarrow$ Infinity

For Each Plan In Plans Do

 Total_Cost $\leftarrow$ 0

 Intermediate_Size $\leftarrow$ 0

 For Each Join In Plan Do

 Input_Size $\leftarrow$ Estimate_Input_Size(Join)

 Output_Size $\leftarrow$ Estimate_Join_Result_Size(Join)

 Io_Cost $\leftarrow$ Estimate_Io_Cost(

 Input_Size,

 Output_Size

)

 Total_Cost $\leftarrow$ Total_Cost + Io_Cost

 Intermediate_Size $\leftarrow$ Output_Size

 End For

 If Total_Cost < Min_Cost Then

 Min_Cost $\leftarrow$ Total_Cost

```
        Best_Plan ← Plan
    End If
End For
Return Best_Plan
End
```

ANALYSIS

Suppose two possible join orders are:

(R1 Join R2) Join R3

R1 Join (R2 Join R3)

Both may produce the same final result, but the size of their intermediate relations can differ. A smaller intermediate relation generally requires fewer disk I/O operations.

RESULT

The join ordering having the minimum estimated I/O cost is selected as the optimal query execution plan.

Q3. NESTED LOOP JOIN

PROBLEM UNDERSTANDING

Nested Loop Join compares each tuple of one relation with every tuple of another relation. It produces a joined relation when the specified join condition is satisfied.

ALGORITHM

Algorithm Nested_Loop_Join(R, S, Condition)

Input: Relations R and S, join condition

Output: Joined relation J

1. Initialize the result relation.
2. Select each tuple from relation R.
3. For every tuple of R, scan all tuples of S.
4. Check the join condition.
5. Add matching tuples to the result.
6. Return the joined relation.

PSEUDOCODE

Algorithm Nested_Loop_Join(R, S, Condition)

Input:

Relation R

Relation S

Join condition

Output:

Joined relation J

Begin

$J \leftarrow \text{Empty}$

For Each Tuple R In R Do

For Each Tuple S In S Do

If Condition(R, S) = True Then

Add Combine(R, S) To J

End If

End For

End For

Return J

End

COMPLEXITY ANALYSIS

Let:

- bR = number of buffer pages of relation R
- bS = number of buffer pages of relation S
- TR = number of tuples in R
- TS = number of tuples in S

For tuple-based Nested Loop Join:

Time Complexity = $O(TR \times TS)$

Every tuple of R may be compared with every tuple of S.

For page-based processing, the approximate I/O cost is:

$bR + (bR \times bS)$

when R is the outer relation.

RESULT

Nested Loop Join successfully produces matching tuples, but its cost increases significantly when the input relations are large.

Q4. DEADLOCK DETECTION USING WAIT-FOR GRAPH

PROBLEM UNDERSTANDING

A Wait-For Graph represents transactions as nodes. An edge from T_i to T_j indicates that T_i is waiting for a resource held by T_j . A cycle in the graph indicates a deadlock.

ALGORITHM

Algorithm Detect_Deadlock(WFG)

Input: Wait-For Graph WFG

Output: Deadlock status

1. Initialize all transactions as unvisited.
2. Initialize the recursion stack.
3. Perform DFS traversal.
4. Check the adjacent transactions.
5. If an unvisited transaction is found, continue DFS.
6. If a transaction already exists in the recursion stack, a cycle exists.
7. Report deadlock if a cycle is detected.
8. Otherwise, report no deadlock.

PSEUDOCODE

Algorithm Detect_Deadlock(WFG)

Input:

Wait-For Graph WFG

Output:

Deadlock status

Begin

For Each Transaction T In WFG Do

 Visited[T] $\leftarrow$ False

 In_Stack[T] $\leftarrow$ False

```

End For

For Each Transaction T In WFG Do

    If Visited[T] = False Then

        If Dfs_Cycle_Check(T) = True Then

            Display "Deadlock Detected"

            Return True

        End If

    End If

End For

Display "No Deadlock"

Return False

```

End

Algorithm Dfs_Cycle_Check(T)

Begin

```

Visited[T] ← True

In_Stack[T] ← True

For Each Transaction U Adjacent To T Do

    If Visited[U] = False Then

        If Dfs_Cycle_Check(U) = True Then

            Return True

        End If

    Else If In_Stack[U] = True Then

        Return True
    End If

End For

```

```
        End If
    End For
    In_Stack[T] ← False
    Return False
End
```

RESULT

The Wait-For Graph is analyzed using DFS. A cycle indicates the presence of a deadlock.

Q5. BASIC TWO-PHASE LOCKING PROTOCOL

PROBLEM UNDERSTANDING

Basic Two-Phase Locking controls concurrent transactions using locks. It consists of a **growing phase**, where locks are acquired, and a **shrinking phase**, where locks are released.

ALGORITHM

Algorithm Basic_2PL(T)

Input: Transaction T

Output: Completed transaction

1. Start the transaction.
2. Request locks for required data items.
3. Grant the lock if it is available.
4. Otherwise, make the transaction wait.
5. Perform the required operations.
6. Enter the shrinking phase after releasing the first lock.
7. Release all remaining locks.
8. Commit the transaction.

PSEUDOCODE

Algorithm Basic_2PL(T)

Input:

Transaction T

Output:

Completed transaction

Begin

For Each Required_Data_Item X In T Do

Request Lock(X)

If Lock(X) Is Available Then

Grant Lock(X)

Else

Wait Until Lock(X) Becomes Available

Grant Lock(X)

End If

End For

For Each Operation Op Of T Do

Execute Op

End For

For Each Lock L Held By T Do

Release L

End For

Commit T

End

IMPORTANT CONSTRAINT

Once a transaction releases its first lock, it cannot acquire any new lock. This separates the growing and shrinking phases.

RESULT

Basic Two-Phase Locking provides controlled concurrent execution and ensures serializable transaction schedules.

Q6. TIMESTAMP-BASED CONCURRENCY CONTROL USING THOMAS' WRITE RULE

PROBLEM UNDERSTANDING

Timestamp-based concurrency control assigns a unique timestamp to every transaction. Each data item maintains read and write timestamps. Thomas' Write Rule allows an obsolete write operation to be ignored instead of aborting the transaction.

ALGORITHM

Algorithm Process_Operation(T, Op, X)

Input: Transaction T, operation Op, data item X

Output: Execute, ignore, or abort

PSEUDOCODE

Algorithm Process_Operation(T, Op, X)

Input:

Transaction T

Operation Op

Data item X

Output:

Operation status

Begin

$Ts \leftarrow \text{Timestamp}(T)$

If Op = Read(X) Then

 If $Ts < \text{Write_Ts}(X)$ Then

 Abort T

 Return

 Else

 Read X

$\text{Read_Ts}(X) \leftarrow \text{Max}(\text{Read_Ts}(X), Ts)$

 End If

Else If Op = Write(X) Then

 If $Ts < \text{Read_Ts}(X)$ Then

 Abort T

 Return

 Else If $Ts < \text{Write_Ts}(X)$ Then

 Ignore Write(X)

 Else

 Write X

$\text{Write_Ts}(X) \leftarrow Ts$

 End If

End If

End

ANALYSIS

The write operation is handled as follows:

$Ts(T) < Read_Ts(X)$

↓

Abort Transaction

$Ts(T) < Write_Ts(X)$

↓

Ignore Obsolete Write

↓

Thomas' Write Rule

Otherwise, the write operation is performed normally.

RESULT

Timestamp ordering maintains concurrency control, while Thomas' Write Rule avoids unnecessary aborts for obsolete write operations.

Q7. ARIES RECOVERY

PROBLEM UNDERSTANDING

ARIES is a recovery technique that restores the database after a system failure. It uses three phases: **Analysis, Redo, and Undo.**

ALGORITHM

Algorithm ARIES_Recovery(Log)

Input: Log records

Output: Consistent database state

1. Perform the Analysis phase.
2. Identify active transactions and dirty pages.
3. Determine the starting point for Redo.
4. Perform the Redo phase.
5. Restore required changes.
6. Identify incomplete transactions.
7. Perform the Undo phase.
8. Reverse changes made by incomplete transactions.
9. Write recovery completion information.

PSEUDOCODE

Algorithm ARIES_Recovery(Log)

Input:

Log records

Output:

Consistent database state

Begin

/* Analysis Phase */

Read Log From Most_Recent_Checkpoint

Initialize Transaction_Table

Initialize Dirty_Page_Table

For Each Log_Record L In Log Do

Update Transaction_Table Using L

Update Dirty_Page_Table Using L

```

End For

Identify Active_Transactions

Identify Committed_Transactions

Identify Uncommitted_Transactions

/* Redo Phase */

Redo_Point ← Determine_Redo_Point()

For Each Log_Record L From Redo_Point Do

    If L Represents Valid_Update Then

        If Update_Not_Already_Applied(L) Then

            Redo L

        End If

    End If

End For

/* Undo Phase */

For Each Transaction T In Uncommitted_Transactions Do

    Traverse T's Log_Records Backwards

    For Each Update Made By T Do

        Undo Update

        Write Compensation_Log_Record

    End For

End For

Write Recovery_Completion_Record

End

```

RESULT

ARIES restores the database to a consistent state by analyzing the log, reapplying required operations, and undoing incomplete transactions.

Q8. SHADOW PAGING RECOVERY

PROBLEM UNDERSTANDING

Shadow Paging maintains a shadow page table and a current page table. Updates are made using new pages. On successful completion, the current page table becomes the active table. On failure, the shadow page table is restored.

ALGORITHM

Algorithm Shadow_Paging

Input: Database pages

Output: Recovered database

1. Create the shadow page table.
2. Create the current page table.
3. Perform updates using newly allocated pages.
4. Update the current page table.
5. If the transaction commits, make the current table active.
6. If failure occurs, restore the shadow table.
7. Complete recovery.

PSEUDOCODE

Algorithm Shadow_Paging

Input:

Database pages

Output:

Recovered database

Begin

Shadow_Table $\leftarrow$ Copy(Current_Page_Table)

Current_Table $\leftarrow$ Copy(Shadow_Table)

While Transaction Is Active Do

 Receive Write(Page, New_Data)

 New_Page $\leftarrow$ Allocate_New_Page()

 Write New_Data To New_Page

 Current_Table[Page] $\leftarrow$ New_Page

End While

If Transaction Commits Then

 Update Database_Root

 Database_Root $\leftarrow$ Current_Table

 Delete Old_Pages When Safe

 Display "Commit Successful"

Else

 Current_Table $\leftarrow$ Shadow_Table

 Database_Root $\leftarrow$ Shadow_Table

 Display "Rollback Completed"

End If

End

RESULT

Shadow Paging allows the database to roll back to its previous consistent state by restoring the shadow page table after a failure.

Q9. IMMEDIATE UPDATE RECOVERY USING UNDO/REDO

PROBLEM UNDERSTANDING

In immediate update recovery, changes may reach the database before a transaction commits. Therefore, log records are used to REDO committed transactions and UNDO incomplete transactions after a crash.

ALGORITHM

Algorithm Immediate_Update_Recovery(Log)

Input: Log file after crash

Output: Consistent database

1. Read the log.
2. Identify committed transactions.
3. Perform REDO for committed transactions.
4. Identify incomplete transactions.
5. Perform UNDO for incomplete transactions.
6. Complete recovery.

PSEUDOCODE

Algorithm Immediate_Update_Recovery(Log)

Input:

Log file after crash

Output:

Consistent database

Begin

Committed $\leftarrow$ Empty

Uncommitted $\leftarrow$ Empty

For Each Log_Record L In Log Do

 If L Is Commit_Record Then

 Add Transaction(L) To Committed

 End If

End For

/* Redo Phase */

For Each Log_Record L In Log From Beginning Do

 T $\leftarrow$ Transaction(L)

 If T In Committed Then

 If L Is Update_Record Then

 Redo L

 End If

 End If

End For

/* Undo Phase */

For Each Log_Record L In Log From End To Beginning Do

 T $\leftarrow$ Transaction(L)

 If T Not In Committed Then

 If L Is Update_Record Then

 Undo L

```
        End If
    End If
End For
Display "Recovery Completed"
End
```

RESULT

Committed transactions are restored using REDO, while incomplete transactions are reversed using UNDO, producing a consistent database state.

Q10. DATABASE CONNECTIVITY MANAGER

PROBLEM UNDERSTANDING

A database connectivity manager handles connection pooling, query execution, transaction management, and exception rollback. Connection pooling allows existing connections to be reused, reducing the overhead of repeatedly creating new connections.

ALGORITHM

Algorithm Database_Connectivity_Manager(Query)

Input: SQL query

Output: Query result or rollback status

1. Check the connection pool.
2. Obtain an available connection or create a new one.
3. Begin a transaction.
4. Execute the query.
5. If successful, commit the transaction.

6. Return the connection to the pool.
7. If an error occurs, rollback the transaction.
8. Return the connection to the pool.
9. Display the error message.

PSEUDOCODE

Algorithm Database_Connectivity_Manager(Query)

Input:

SQL query

Output:

Query result

Begin

If Available_Connection_Exists() Then

 Connection ← Get_Connection_From_Pool()

Else

 Connection ← Create_New_Connection()

End If

Try

 Begin_Transaction(Connection)

 Result ← Execute_Query(Connection, Query)

 If Query_Is_Successful(Result) Then

 Commit_Transaction(Connection)

 Return_Connection_To_Pool(Connection)

 Return Result

 Else

```
        Rollback_Transaction(Connection)

        Return_Connection_To_Pool(Connection)

        Display "Query Failed"

    End If

Catch Exception E

    Rollback_Transaction(Connection)

    Return_Connection_To_Pool(Connection)

    Display "Database Error"

End Try

End
```

RESULT

The database connectivity manager efficiently manages connections, executes queries, commits successful transactions, and rolls back failed transactions, thereby improving reliability and resource utilization.